



Instituto Tecnológico de Estudios Superiores de Monterrey
Campus Monterrey

Escuela de Ingeniería y Ciencias
Ingeniería en Ciencias de Datos y Matemáticas

Uso de álgebras modernas para seguridad y criptografía (Gpo 603)

Profesores: Raúl Gómez Anas Wajid y Alberto F. Martínez

Socio Formador: Casa Monarca

Documento SDK

Equipo 1

Miranda Cornejo Ahuja - A01571668

Natalia Cervantes González - A00838465

Hugo Jaime Valdés - A01384039

Jorge Alfonso Andujo Valdéz - A01563535

Ana Sofía Ibarra Ponce - A01285836

ÍNDICE

1. DESCRIPCIÓN GENERAL.....	3
2. ARQUITECTURA DEL SISTEMA.....	4
3. STACK TECNOLÓGICO.....	6
4. ESTRUCTURA DEL PROYECTO.....	7
5. CONFIGURACIÓN DEL ENTORNO.....	9
6. INSTALACIÓN Y EJECUCIÓN.....	10
7. BASE DE DATOS.....	12
8. API / INTERFACES.....	16
9. FLUJOS Y LÓGICA DE NEGOCIO.....	18
10. EJEMPLOS DE USO.....	20
11. MANEJO DE ERRORES.....	21
12. PRUEBAS.....	22
13. DESPLIEGUE.....	23
14. SEGURIDAD.....	20
15. GUÍA DE CONTRIBUCIÓN.....	25
16. PROBLEMAS CONOCIDOS Y LIMITACIONES.....	28
17. CASOS DE USO PRINCIPALES.....	31
18. FAQ TÉCNICA Y REGISTRO DE ERRORES (LOGS).....	37

1. Descripción General

1.1 Propósito del sistema

Casa Monarca — Identity Access Manager es un sistema web para gestión de identidades digitales, control de acceso basado en roles (RBAC), registro de personas migrantes con integridad criptográfica, y firma documental con certificados PKI. Fue construido como respuesta a un reto académico enfocado en "Firmas solidarias y protección de datos personales".

1.2 Alcance

Permite a una organización de atención a migrantes (Casa Monarca) registrar personas de forma segura, con trazabilidad total de quién hizo qué y cuándo, protección criptográfica de los datos sensibles, autenticación multifactor obligatoria, y firma digital de documentos con un modelo similar al SAT mexicano.

1.3 Usuarios principales

La siguiente tabla describe los cuatro niveles de acceso del sistema, el rol asociado a cada nivel y los permisos que se otorgan a cada uno.

Nivel	Rol	Permisos
1	Administrador del Sistema	CRUD completo, gestión de usuarios, bitácora, plantillas
2	Coordinador de Área	Crear / Leer / Editar registros. Sin eliminar ni gestionar usuarios
3	Personal Operativo	Crear / Leer registros. Sin editar ni eliminar
4	Personal Externo (becarios, voluntarios)	Solo crear registros. Sin lectura de registros internos

1.4 Modulos principales

- Autenticación — JWT de dos pasos con TOTP obligatorio (RFC 6238)
- Gestión de Identidades — Alta, activación por token, vigencia, niveles, revocación
- Registros de Migrantes — CRUD con folio autogenerated, hash SHA-256 y cifrado de campos sensibles
- Certificados Criptográficos — RSA-2048 de identidad + ECDSA P-256 de firma documental (modelo SAT)

- Firma Documental — Flujo .cer/.key con Web Crypto API en el navegador
- Plantillas de Captura — Formularios configurables por administrador
- Bitácora de Auditoría — Log inmutable de todas las acciones críticas
- Notificaciones — SMTP o modo demo (cola en memoria)

1.5 Casos de uso NO implementados

La siguiente tabla lista las funcionalidades identificadas durante el diseño del sistema que no están implementadas en la versión actual, indicando su estado y el impacto operativo que genera su ausencia.

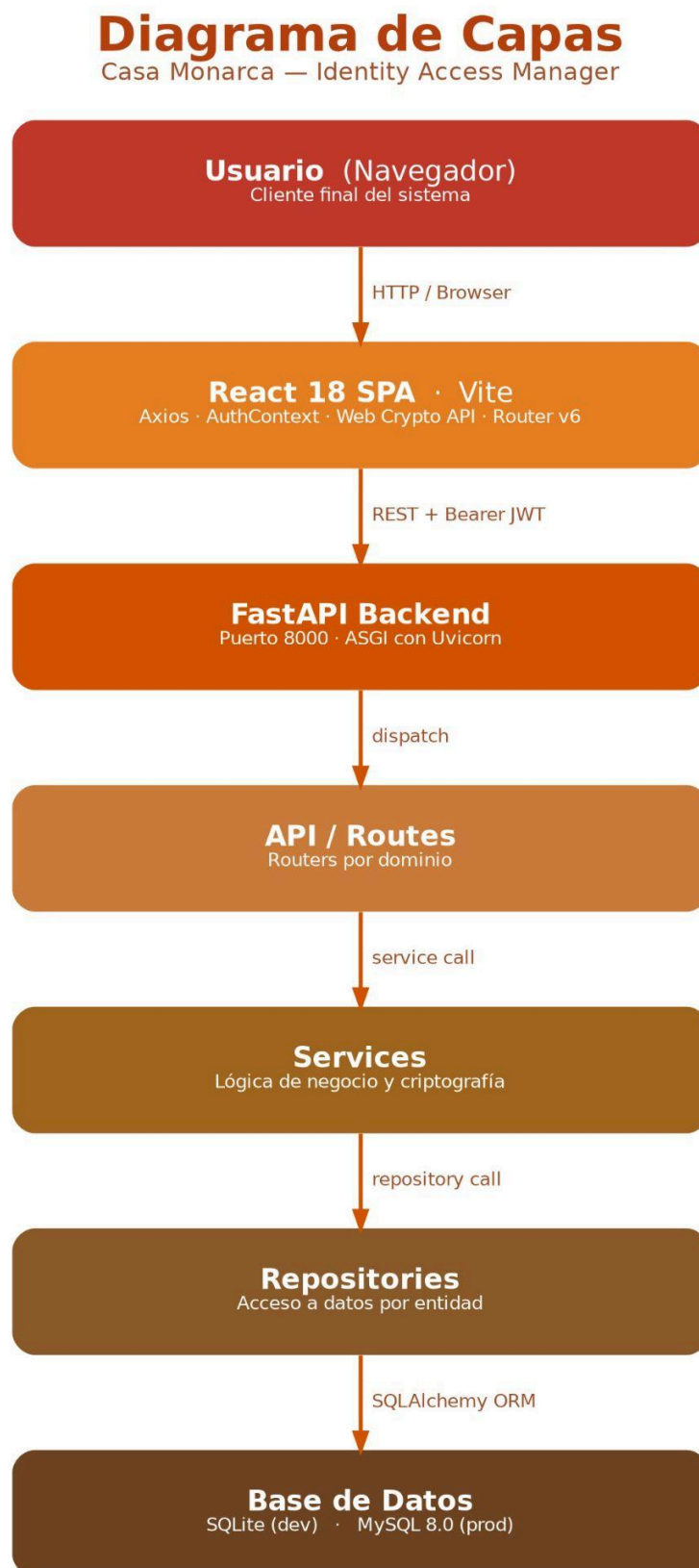
Caso de uso	Estado	Impacto operativo
Verificación pública de folio	Placeholder vacío	El ciudadano/migrante no puede verificar su registro externamente
Módulo de documentos adjuntos	Placeholder vacío	No se puede adjuntar documentación a un registro
Recuperación de contraseña olvidada	Ausente	Requiere intervención manual del admin para revocar y re-activar la cuenta
Exportación de registros (PDF/Excel)	Ausente	Los reportes deben hacerse directamente sobre la BD
Dashboard estadístico KPIs	Parcial — solo conteos básicos	Coordinadores y operativos tienen bandeja pero sin métricas
Soporte multi-organización	Ausente (single-tenant)	El sistema solo soporta una organización a la vez
Notificación de expiración próxima	Ausente	Cuentas y certificados expiran sin previo aviso automático

2. Arquitectura Del Sistema

2.1 Tipo de arquitectura

El sistema sigue una arquitectura de monolito modular con separación física de frontend y backend. No utiliza Docker. El backend puede alojarse en cualquier servidor Python y el frontend se sirve como SPA estático.

2.2 Diagrama de capas



2.3 Servicios internos

La siguiente tabla enumera los servicios internos del backend y la responsabilidad principal de cada uno dentro de la arquitectura del sistema.

Servicio	Responsabilidad
CryptoService	Certificados RSA-2048 de identidad
SigningService	Certificados ECDSA P-256 (modelo SAT), cifrado .key
TOTPService	TOTP RFC 6238 + generación de QR code
FieldEncryptionService	Cifrado Fernet sobre campos sensibles
AuditService	Log inmutable de acciones críticas
NotificationService	Envío SMTP o modo demo
AuthService	Login, validación de estados, JWT pre-2FA y final
IdentityService	Ciclo de vida de usuarios: alta, activación, cambio de nivel, revocación
RecordService	CRUD de registros, hash SHA-256, workflow operativo
ArcoService	Solicitudes ARCO, anonimización controlada
DeletionRequestService	Solicitudes de eliminación con aprobación del admin
TemplateService	CRUD de plantillas de formulario configurables
PermissionService	Verificación de permisos por código de acción

2.4 Comunicación entre capas

El frontend se comunica exclusivamente con el backend vía HTTP REST sobre VITE_API_URL (por defecto <http://localhost:8000/api/v1>). El JWT se almacena en localStorage con clave `ims_token`; los interceptores Axios lo inyectan en cada request como `Authorization: Bearer <token>`. No hay WebSockets, GraphQL ni colas de mensajes.

2.5 Dependencias externas

La siguiente tabla describe las dependencias externas requeridas por el sistema para su correcto funcionamiento en producción.

Dependencia	Propósito
-------------	-----------

SMTP externo (Gmail, Mailgun, etc.)	Envío de enlaces de activación y códigos OTP
Ninguna otra	El sistema funciona offline con SQLite

3. Stack Tecnológico

3.1 Backend (Python)

La siguiente tabla lista las librerías de Python utilizadas en el backend, su versión requerida y el rol que desempeñan dentro del sistema.

Librería	Versión	Rol
Python	3.10+	Lenguaje principal
FastAPI	0.110.0	Framework API REST
uvicorn[standard]	0.29.0	ASGI server
SQLAlchemy	2.0.29	ORM
pymysql	1.1.0	Driver MySQL
alembic	1.13.1	Migraciones de BD
python-jose[cryptography]	3.3.0	JWT
passlib[bcrypt]	1.7.4	Hash de contraseñas
bcrypt	4.0.1	Backend bcrypt
pydantic	1.10.14 (v1)	Validación de schemas
python-dotenv	1.0.1	Variables de entorno
cryptography	>=42.0.0	RSA-2048, ECDSA P-256, Fernet, X.509, AES-GCM
pyotp	2.9.0	TOTP RFC 6238
qrcode[pil]	7.4.2	QR de enrolamiento TOTP
pytest	8.1.1	Testing
email-validator	2.1.1	Validación de emails en schemas Pydantic
python-multipart	0.0.9	Soporte para OAuth2PasswordRequestForm

3.2 Frontend (JavaScript)

La siguiente tabla lista las librerías de JavaScript utilizadas en el frontend, su versión requerida y la función que cumplen en la aplicación.

Librería	Versión	Rol
React	18.2.0	Framework UI
react-dom	18.2.0	Renderizado DOM
react-router-dom	6.23.1	Enrutamiento SPA
axios	1.7.2	Cliente HTTP
Vite	5.2.11	Build tool y dev server
@vitejs/plugin-react	4.3.0	Plugin Vite para React
Web Crypto API	nativa	Firma ECDSA P-256, AES-GCM, PBKDF2 en navegador

3.3 Motor de base de datos

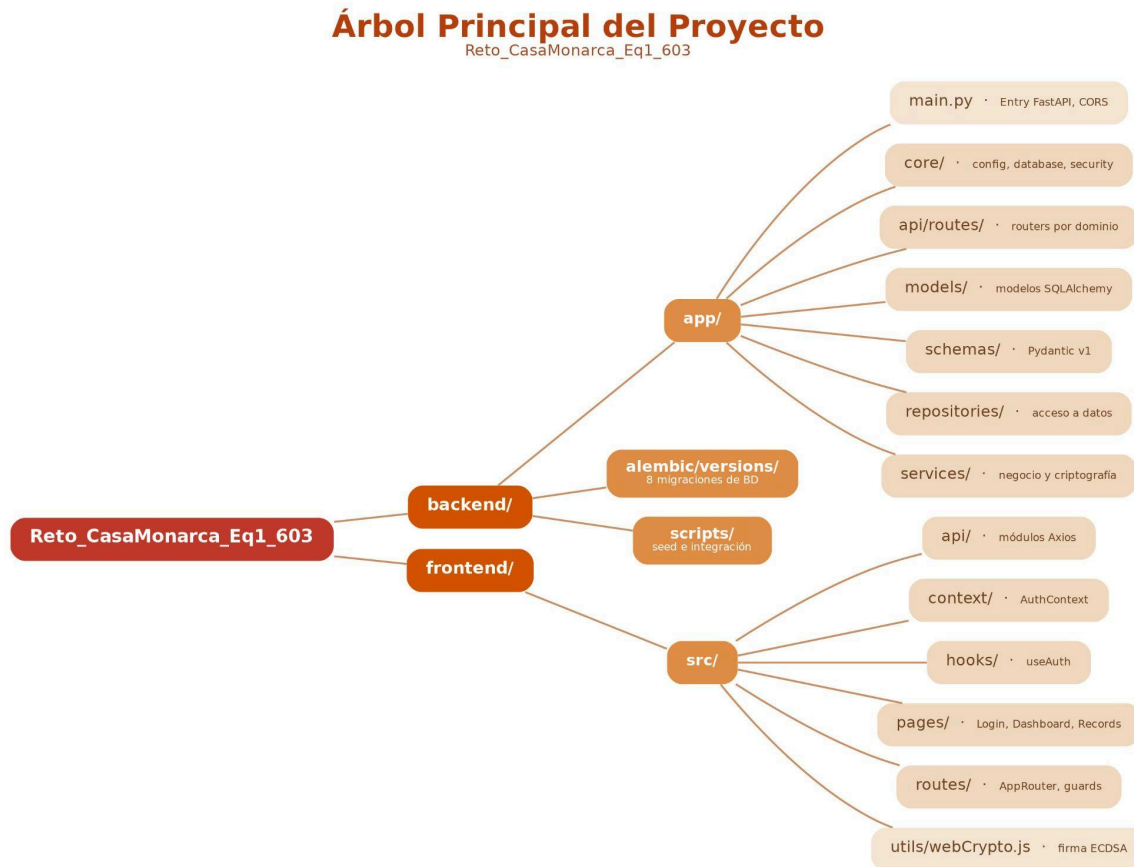
- SQLite — Desarrollo local (por defecto con SQLITE_PATH=./app.db)
- MySQL 8.0 — Producción (vía DATABASE_URL o variables MYSQL_*)

3.4 Herramientas de build / infra

- Sin Docker ni CI/CD configurado
- Alembic para migraciones (8 migraciones actualmente)
- pytest 8.1.1 para pruebas unitarias

4. Estructura Del Proyecto

4.1 Árbol principal



4.2 Routers del backend

La siguiente tabla describe los routers del backend, el archivo donde están definidos y la función que cumplen dentro de la API.

Router	Archivo	Función
/auth	routes/auth.py	Login, TOTP, firma documental
/identity	routes/identity.py	Alta usuarios, activación, vigencia, revocación
/users	routes/users.py	Proxy de listado (retrocompat)
/roles	routes/roles.py	Roles, permisos, niveles

/certificates	routes/certificates.py	Certificados criptográficos
/records	routes/records.py	CRUD registros de migrantes
/templates	routes/templates.py	CRUD plantillas de captura
/audit	routes/audit.py	Bitácora (solo admin)
/documents	routes/documents.py	PLACEHOLDER — sin implementar
/verification	routes/verification.py	PLACEHOLDER — sin implementar
/arco-requests	routes/arco_requests.py	CRUD solicitudes ARCO, escalamiento, anonimización
/deletion-requests	routes/deletion_requests.py	Solicitudes de eliminación con aprobación admin
/dashboard	routes/dashboard.py	Estadísticas y bandeja de trabajo por rol
/signatures	routes/signatures.py	Consulta y verificación de firmas documentales

5. Configuración Del Entorno

5.1 Variables de entorno - Backend (backend/.env)

```
# App
APP_NAME=Casa Monarca Identity Manager
API_V1_PREFIX=/api/v1

# Seguridad
SECRET_KEY=CHANGE_THIS_TO_A_RANDOM_64_CHAR_HEX
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60

# Base de datos (SQLite dev)
SQLITE_PATH=./app.db
AUTO_CREATE_TABLES=true

# Base de datos (MySQL prod)
# DATABASE_URL=mysql+pymysql://user:password@host:3306/dbname

# SMTP (vacío = modo demo)
SMTP_HOST=
SMTP_PORT=587
SMTP_USER=
SMTP_PASSWORD=
SMTP_FROM_EMAIL=no-reply@demo.org

# Cifrado de campos sensibles (Fernet)
```

FIELD_ENCRYPTION_KEY=

Admin por defecto (solo para seed)

ADMIN_EMAIL=admin@demo.org

ADMIN_PASSWORD=Admin123!

CORS

CORS_ORIGINS=http://localhost:5173,http://127.0.0.1:5173

5.2 Variables de entorno - Frontend (frontend/.env)

VITE_API_URL=http://localhost:8000/api/v1

5.3 Puertos y URLs locales

La siguiente tabla especifica los puertos y URLs locales utilizados por cada servicio del sistema en el entorno de desarrollo.

Servicio	Puerto	URL
Frontend (dev)	5173	http://localhost:5173
Backend API	8000	http://localhost:8000
Swagger UI	8000	http://localhost:8000/docs
ReDoc	8000	http://localhost:8000/redoc
Health check	8000	http://localhost:8000/health

5.4 Configuración adicionales

- ADMIN_FULL_NAME="Administrador del Sistema" — nombre del admin creado en el seed
- TOTP_ENROLL_TOKEN_EXPIRE_MINUTES=10 — vida del token durante enrolamiento TOTP
- PRE2FA_TOKEN_EXPIRE_MINUTES=10 — vida del JWT temporal pre-2FA
- ACTIVATION_URL_BASE=http://localhost:5173/activate — base URL del enlace
- ACTIVATION_TOKEN_EXPIRE_HOURS=48 — vigencia del enlace de activación (default 48 h)
- Variables adicionales en backend/.env:
 - o backend/app/core/config.py — la instancia settings se crea con @lru_cache(), garantizando una única instancia por proceso.
 - o frontend/vite.config.js — el servidor de desarrollo escucha en host: "0.0.0.0" (accesible desde red local) en el puerto 5173.

- o backend/alembic.ini — sqlalchemy.url apunta a SQLite hardcodeado, pero alembic/env.py lo sobrescribe con settings.resolved_database_url en tiempo de ejecución.

6. Instalación y Ejecución

6.1 Backend

```
# 1. Ir al directorio backend
cd bac+kend

# 2. Crear entorno virtual
python -m venv .venv
.venv\Scripts\activate          # Windows
# source .venv/bin/activate      # Linux/Mac

# 3. Instalar dependencias
pip install -r requirements.txt

# 4. Configurar entorno
copy .env.example .env          # Windows
# cp .env.example .env          # Linux/Mac

# 5. Levantar la API
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

6.2 Seed inicial (con servidor corregido)

```
cd backend
python -m scripts.seed_initial_data
```

El seed crea: 4 niveles de acceso, 8 permisos base, 4 roles (SYSTEM_ADMIN, AREA_COORDINATOR, AREA_OPERATOR, EXTERNAL_STAFF), 5 áreas, 9 usuarios activos, 5 registros de migrantes demo y 1 plantilla base.

6.3 Migraciones con Alembic

```
cd backend
alembic upgrade head          # Aplicar migraciones
alembic current               # Ver versión actual
alembic revision --autogenerate -m 'descripcion' # Nueva migración
alembic downgrade -1          # Revertir una migración
```

Nota: Las migraciones 0002 y 0003 usan PRAGMA table_info específico de SQLite. En MySQL deben modificarse.

6.4 Frontend

```
cd frontend
npm install
copy .env.example .env          # Windows / cp .env.example .env (Linux)
npm run dev                     # Servidor desarrollo
npm run build                   # Build producción
npm run preview                 # Preview del build
```

6.5 Problemas conocidos de instalación

La siguiente tabla documenta los problemas más comunes que pueden surgir durante la instalación del sistema y sus causas principales.

Problema	Causa
bcrypt falla en Windows con Python 3.12+	Conflicto de ABI entre passlib y bcrypt
qrcode ImportError: No module named PIL	qrcode[pil] requiere Pillow
Warning SECRET_KEY al arrancar	Valor default detectado en config.py
Frontend "Network Error" / "CORS policy"	VITE_API_URL o CORS_ORIGINS incorrectos
alembic: Can't locate revision	BD en estado inconsistente o reset manual

6.6 Credenciales demo tras el seed

Nota: cuentas demo creadas con status=ACTIVE y totp_enrolled=False. El primer login solicitará enrolamiento TOTP.

La siguiente tabla muestra las credenciales de acceso de los usuarios de prueba generados por el seed inicial. Todos los usuarios requerirán enrolamiento TOTP en su primer inicio de sesión.

Email	Contraseña	Nivel
admin@demo.org	Admin123! (ver ADMIN_PAS SWORD en .env)	1 — Administrador

coordinador@demo.org	CoordHuman 2026!	2 — Coordinador
operador@demo.org	Oper2026Seguro!	3 — Operativo
voluntario@demo.org	Voluntario2026!	4 — Externo

7. Base de Datos

7.1 Motor y ORM

Desarrollo: SQLite (./app.db en backend/). Producción: MySQL 8.0. ORM: SQLAlchemy 2.0.29 con patrón declarative_base. Schemas de validación: Pydantic v1.

7.2 Migraciones existentes

La siguiente tabla describe el historial de migraciones de base de datos aplicadas al proyecto, indicando su identificador, fecha de creación y el contenido principal de cada migración.

ID	Fecha	Contenido
20260410_0001	2026-04-10	Esquema inicial completo
20260421_0002	2026-04-21	TOTP en credentials, cert_type en certificates, tabla document_signatures, columnas cifradas en migrant_records
0003_signing_sat	2026-04-24	Columna revoked_at en certificates
20260522_0004	2026-05-22	workflow_status, assigned_operator/coordinator_id, channeled_at, reviewed_at en migrant_records
20260526_0005	2026-05-26	Tablas arco_requests y deletion_requests; campo is_anonymized en migrant_records
20260529_0006	2026-05-29	Columna channel_notes en migrant_records
20260529_0007	2026-05-29	Columna matricula en users; folio en migrant_records, arco_requests y deletion_requests
20260529_0008	2026-05-29	Columna folio en audit_logs + backfill de registros existentes (BIT-NNNNN)

7.3 Tablas principales

users

La siguiente tabla describe las columnas de la tabla *users*, que almacena la información principal de los colaboradores y administradores del sistema.

Columna	Tipo	Notas
id	INTEGER PK	
full_name	VARCHAR(150)	
email	VARCHAR(190) UNIQUE	
status	VARCHAR(20)	PENDING, ACTIVE, INACTIVE, REVOKED
area_id	FK → areas	
access_level_id	FK → access_levels	
role_id	FK → roles	
starts_at / expires_at	DATETIME	Vigencia del usuario
matricula	VARCHAR(20) UNIQUE	Formato CM-USR-NNNN, nullable
is_active	BOOLEAN	False hasta activación
created_by_id	FK → users	Admin que creó el usuario
user_subtype	VARCHAR(30)	Nullable, subtipo de usuario
assigned_to_id	FK → users	Supervisor jerárquico (self-referencia)
created_at	DATETIME	Timestamp de creación
updated_at	DATETIME	Timestamp de última modificación

credentials

La siguiente tabla describe las columnas de la tabla *credentials*, encargada de almacenar las credenciales de autenticación (contraseña y TOTP) de cada usuario.

Columna	Tipo	Notas
user_id	FK → users UNIQUE	Relación 1:1 con user
password_hash	VARCHAR(255)	bcrypt
totp_secret_enc	VARCHAR(512)	Secreto TOTP cifrado con Fernet
totp_enrolled	BOOLEAN	

certificates

La siguiente tabla describe las columnas de la tabla *certificates*, que almacena los certificados criptográficos de identidad (RSA-2048) y de firma (ECDSA P-256) de los usuarios.

Columna	Tipo	Notas
serial_number	VARCHAR(120) UNIQUE	UUID hex
cert_type	VARCHAR(20)	IDENTITY (RSA-2048) o SIGNING (ECDSA P-256)
status	VARCHAR(40)	VALID, REVOKED, EXPIRED
fingerprint	VARCHAR(64)	SHA-256 del JSON de cert_data
cert_data	TEXT	JSON con clave pública PEM + metadata

migrant_records

La siguiente tabla describe las columnas de la tabla *migrant_records*, que almacena los registros de personas migrantes atendidas por la organización, incluyendo datos cifrados e integridad SHA-256.

Columna	Tipo	Notas
folio	VARCHAR(30) UNIQUE	Formato CM-YYYY-NNNN
name_or_alias	VARCHAR(200)	En claro (para búsqueda)
name_or_alias_enc	TEXT	Cifrado con Fernet
contact_info / contact_info_enc	VARCHAR / TEXT	En claro y cifrado
sha256_hash	VARCHAR(64)	Hash de integridad del registro
status	VARCHAR(30)	REGISTRADO, EN_PROCESO, ATENDIDO, CERRADO
workflow_status	VARCHAR(20)	pendiente → canalizado → revisado → cerrado
assigned_operator_id	FK → users	Operativo responsable del registro

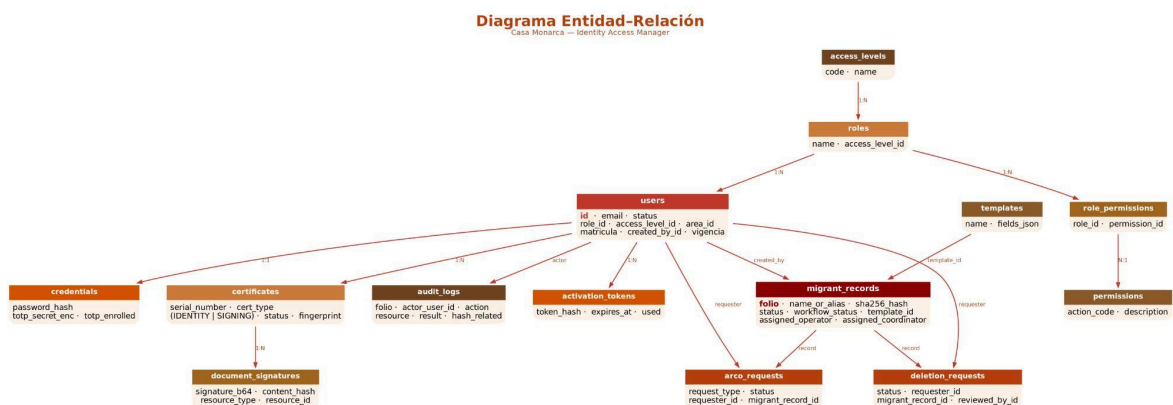
assigned_coordinator_id	FK → users	Coordinador al que fue canalizado
channeled_at	DATETIME	Timestamp de canalización
channel_notes	TEXT	Notas del operativo al canalizar
reviewed_at	DATETIME	Timestamp de revisión
is_anonymized	BOOLEAN	True tras aprobación de ARCO de cancelación
anonymized_at	DATETIME	Timestamp de anonimización

audit_logs

La siguiente tabla describe las columnas de la tabla `audit_logs`, que almacena el registro inmutable de todas las acciones críticas realizadas en el sistema.

Columna	Tipo	Notas
actor_user_id	FK → users (nullable)	Null en acciones anónimas
action	VARCHAR(120)	ej: auth.login, records.create
resource	VARCHAR(120)	ej: user, migrant_record
result	VARCHAR(20)	SUCCESS, FAILURE
hash_related	VARCHAR(64)	Hash/fingerprint relevante

7.4 Diagrama ER (simplificado)



8. API / Interfaces

Base URL: `http://localhost:8000/api/v1`

Autenticación: Bearer JWT en header Authorization. El login es de dos pasos obligatorio.

8.1 Endpoints de autenticación (/auth)

La siguiente tabla describe los endpoints disponibles en el módulo de autenticación, incluyendo el método HTTP, la ruta, la descripción de la operación y el tipo de autenticación requerida.

Método	Endpoint	Descripción	Auth
GET	/health	Health check básico	No
POST	/auth/login	Paso 1: valida credenciales, retorna token pre-2FA	No
POST	/auth/verify-2fa	Paso 2: valida TOTP, retorna JWT final	No
GET	/auth/me	Información del usuario autenticado	JWT
GET	/auth/demo/activation-links	Links de activación en modo demo	Admin
GET	/auth/totp/status	Estado de enrolamiento TOTP	JWT
POST	/auth/totp/enroll/begin	Genera secreto TOTP y QR	JWT
POST	/auth/totp/enroll/confirm	Confirma enrolamiento TOTP	JWT
DELETE	/auth/totp/revoke/{user_id}	Revoca TOTP de un usuario	JWT
GET	/auth/signing/status	Estado del certificado de firma ECDSA	JWT
POST	/auth/signing/enroll	Genera par ECDSA P-256 + certificado SAT	JWT
GET	/auth/signing/certificate/download	Re-descarga el .cer del usuario	JWT
POST	/auth/signing/verify	Verifica firma ECDSA enviada desde navegador	JWT
GET	/auth/signing/signatures/{type}/{id}	Lista firmas de un recurso	JWT
DELETE	/auth/signing/revoke/{user_id}	Admin revoca certificado de firma	Admin

8.2 Endpoints de identidad (/identity)

La siguiente tabla describe los endpoints del módulo de gestión de identidades, que permiten administrar el ciclo de vida de los usuarios del sistema (alta, activación, revocación, certificados).

Método	Endpoint	Descripción	Auth
POST	/identity/collaborators	Crea usuario, genera activación y cert RSA-2048	Admin
POST	/identity/activate	Activa cuenta con token y password inicial	No
GET	/identity/users	Lista todos los usuarios	Admin
GET	/identity/users/{id}	Detalle de un usuario	Admin
PATCH	/identity/users/{id}/level	Cambia nivel, rol y/o área	Admin
PATCH	/identity/users/{id}/status	Desactiva un usuario	Admin
PATCH	/identity/users/{id}/revoke	Revoca identidad (REVOKED + revocar certs)	Admin
PATCH	/identity/users/{id}/reactivate	Reactiva y reemite certificado	Admin
PATCH	/identity/users/{id}/expiry	Actualiza vigencia y reemite cert	Admin
GET	/identity/users/{id}/certificate	Certificado RSA-2048 del usuario	Admin
POST	/identity/users/{id}/certificate/reissue	Revoca y reemite certificado	Admin
GET	/identity/areas	Lista áreas activas	Admin

8.3 Endpoints de Registros (/records)

La siguiente tabla describe los endpoints del módulo de registros de migrantes, indicando las operaciones disponibles y el nivel de acceso requerido para cada una.

Método	Endpoint	Descripción	Auth
POST	/records/	Crear registro de migrante con folio + SHA-256	Niveles 1-4
GET	/records/	Lista registros con filtros (area_id, status, search)	Niveles 1-3
GET	/records/{id}	Detalle de un registro	Niveles 1-3
PATCH	/records/{id}	Editar registro, recalcula SHA-256	Niveles 1-2
DELETE	/records/{id}	Hard delete (irreversible)	Admin

GET	/records/{id}/hash	Consulta hash SHA-256 de integridad	Niveles 1-3
PUT	/records/{id}/review	Aprobar registro (requiere firma para niveles 1-2)	Niveles 1-3
PUT	/records/{id}/channel	Canalizar registro del operativo al coordinador	Nivel 3
PUT	/records/{id}/close	Cerrar registro (workflow_status = cerrado)	Niveles 1-2

8.4 Otros endpoints

La siguiente tabla resume los demás módulos de la API con su prefijo de ruta y una descripción de su funcionalidad general.

Prefijo	Descripción
/roles/	Lista roles con permisos (cualquier nivel autenticado)
/roles/permissions	Lista todos los permisos disponibles
/roles/levels	Lista niveles de acceso
/certificates/	Lista todos los certificados (admin)
/certificates/validate/{id}	Valida estado de un certificado
/templates/	CRUD de plantillas de captura (admin)
/audit/	Bitácora de auditoría (admin)
/documents/placeholder	STUB — no implementado
/verification/placeholder	STUB — no implementado
/arco-requests/	CRUD ARCO, escalamiento (coordinador), revisión y anonimización (admin)
/deletion-requests/	Solicitar eliminación (niveles 1-2), aprobar/rechazar (solo admin)
/dashboard/	Estadísticas y bandeja de trabajo personalizada por rol

9. Flujos y Lógica de Negocio

9.1 Flujo de autenticación (dos pasos)

- Paso 1: El usuario envía credenciales a POST /auth/login y recibe un session_token (JWT pre-2FA) con vigencia de 10 minutos.

- Paso 2: El usuario envía el session_token junto con el código TOTP a POST /auth/verify-2fa y recibe el JWT definitivo.
- Si el usuario no tiene TOTP enrolado (primer login), el flujo redirige al enrolamiento antes de continuar.
- El JWT final se almacena en localStorage (clave ims_token) y los interceptores Axios lo adjuntan en cada request.

9.2 Alta de usuarios

- El administrador crea un usuario en POST /identity/collaborators con nivel, rol, área y vigencia.
- El sistema genera automáticamente un token de activación (SHA-256 almacenado), emite un certificado RSA-2048 y envía el enlace de activación por email (o lo almacena en memoria en modo demo).
- El usuario accede al enlace y establece su contraseña inicial en POST /identity/activate.
- Tras la activación, el usuario debe enrolarse en TOTP en su primer login.

9.3 Flujo de registro de migrante

- Cualquier nivel autenticado (1-4) puede crear un registro en POST /records/.
- El sistema genera automáticamente el folio en formato CM-YYYY-NNNN.
- Se calcula un hash SHA-256 de los campos del registro para garantizar integridad.
- Si FIELD_ENCRYPTION_KEY está configurado, name_or_alias y contact_info se cifran con Fernet y se almacenan también en campos _enc.
- Solo niveles 1-2 pueden editar; solo nivel 1 puede eliminar (hard delete irreversible).

9.4 Flujo de firma documental (modelo SAT)

- El usuario genera su par ECDSA P-256 en POST /auth/signing/enroll. El backend genera la clave privada cifrada (.key) y el certificado (.cer); ambos se entregan UNA SOLA VEZ. La clave privada NO se persiste en el backend.
- Para firmar, el usuario carga el .cer y el .key en el navegador; la Web Crypto API (SubtleCrypto) descifra la clave privada y firma el hash del recurso en el cliente.
- La firma base64 y el hash del contenido se envían al backend en POST /auth/signing/verify para registro en document_signatures.

9.5 Control de acceso (RBAC)

Los endpoints verifican el nivel de acceso mediante dependency injection (FastAPI deps). El campo access_level_code del JWT determina los permisos: nivel 1 tiene acceso total, cada nivel inferior restringe las operaciones disponibles sobre registros, usuarios y certificados.

10. Ejemplos de Uso

10.1 Login completo (dos pasos)

Paso 1 - Enviar credenciales

```
POST /api/v1/auth/login
Content-Type: application/x-www-form-urlencoded

username=admin@demo.org&password=Admin123!

Respuesta 200:
{
  "requires_2fa": true,
  "session_token": "<jwt_pre_2fa>",
  "two_fa_type": "totp",
  "requires_totp_enrollment": false
}
```

Paso 2 - Validar TOTP

```
POST /api/v1/auth/verify-2fa
Content-Type: application/json

{
  "session_token": "<jwt_pre_2fa>",
  "otp_code": "123456"
}

Respuesta 200:
{
  "access_token": "<jwt_authenticated>",
  "token_type": "bearer"
}
```

10.2 Crear un registro de migrante

```
POST /api/v1/records/
Authorization: Bearer <jwt_authenticated>

{
  "name_or_alias": "Juan Pérez",
  "nationality": "Honduras",
  "language": "Español",
  "age_range": "25-34",
  "gender": "Masculino",
  "contact_info": "555-1234",
  "needs": ["alimentación", "alojamiento"],
  "registration_date": "2026-04-28T00:00:00",
  "observations": "Llegó en buen estado de salud."
}
```

```
"area_id": 4,  
"status": "REGISTRADO",  
"template_id": 1  
}
```

Respuesta 201:

```
{ "folio": "CM-2026-0006", "sha256_hash": "a3f...", ... }
```

10.3 Crear un nuevo colaborador

POST /api/v1/identity/collaborators

Authorization: Bearer <jwt_admin>

```
{  
  "full_name": "María García",  
  "email": "maria@demo.org",  
  "area_id": 2,  
  "access_level_code": 3,  
  "role_id": 3,  
  "starts_at": "2026-04-28T00:00:00",  
  "expires_at": "2027-04-28T00:00:00"  
}
```

11. Manejo de Errores

11.1 Códigos HTTP utilizados

La siguiente tabla describe los códigos de respuesta HTTP utilizados por la API y las situaciones en las que cada uno es retornado al cliente.

Código	Situación
200 OK	Operación exitosa
201 Created	Recurso creado correctamente (registros de migrantes)
400 Bad Request	Datos inválidos, token vencido o código TOTP incorrecto
401 Unauthorized	Credenciales inválidas, JWT ausente/inválido, usuario inactivo
403 Forbidden	Nivel de acceso insuficiente para la operación
404 Not Found	Recurso no encontrado
422 Unprocessable Entity	Error de validación Pydantic en el body
500 Internal Server Error	Error inesperado del servidor
409 Conflict	Email o folio duplicado; recurso único en conflicto

11.2 Formato de error estándar (FastAPI)

```
{
  "detail": "Mensaje descriptivo del error"
}

// En errores 422:
{
  "detail": [
    { "loc": ["body", "email"], "msg": "field required", "type": "value_error.missing" }
  ]
}
```

11.3 Errores comunes y soluciones

La siguiente tabla documenta los errores más frecuentes que pueden presentarse durante el uso del sistema, junto con su causa probable y la solución recomendada.

Error	Causa probable	Solución
401 en login	Contraseña incorrecta o usuario inactivo/revocado	Verificar credenciales o estado del usuario en admin
401 en verify-2fa	Código TOTP incorrecto o expirado (>30s)	Reintentar con código vigente o resincronizar el reloj
400 en activate	Token de activación expirado (>48h) o ya consumido	El admin debe revocar y crear nuevo usuario
403 en records DELETE	El usuario no es nivel 1 (admin)	Solo administradores pueden eliminar registros
500 en signing/enroll	FIELD_ENCRYPTION_KEY no configurado	Definir la variable en el .env del backend

12. Pruebas

12.1 Pruebas unitarias

El proyecto incluye pytest 8.1.1 como framework de pruebas. Actualmente existe un test unitario formal:

- backend/tests/test_permission_service.py — Valida la lógica del servicio de permisos.

12.2 Scripts de integración manual

- backend/scripts/test_demo.py — Test completo del flujo de autenticación y registros contra el servidor local.

- `backend/scripts/verify_endpoints.py` — Verifica que todos los endpoints respondan correctamente.

12.3 Ejecución de pruebas

```
cd backend
.venv\Scripts\activate # activar entorno virtual
pytest tests/ -v        # correr tests unitarios
python -m scripts.test_demo # test de integración manual
python -m scripts.verify_endpoints # verificación de endpoints
```

Nota: La cobertura de pruebas es mínima. Se recomienda ampliar los tests unitarios para services, repositories y la lógica criptográfica antes de producción.

13. Despliegue

13.1 Prerrequisitos de producción

- Python 3.10+ instalado en el servidor
- Node.js 18+ para el build del frontend
- MySQL 8.0 como base de datos
- Servidor SMTP externo (Gmail, Mailgun, SendGrid, etc.)
- Dominio y certificado TLS/HTTPS para el backend

13.2 Pasos de despliegue del backend

```
# 1. Clonar el repositorio y configurar entorno
git clone <repo_url> && cd backend
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# 2. Configurar .env de producción
DATABASE_URL=mysql+pymysql://user:pass@host:3306/dbname
SECRET_KEY=<hex_aleatorio_64_chars>
FIELD_ENCRYPTION_KEY=<fernet_key>
SMTP_HOST=smtp.gmail.com
CORS_ORIGINS=https://tudominio.com

# 3. Aplicar migraciones
alembic upgrade head

# 4. Ejecutar seed inicial
python -m scripts.seed_initial_data

# 5. Levantar con uvicorn (o usar gunicorn + uvicorn workers)
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

13.3 Build del frontend

```
cd frontend
echo 'VITE_API_URL=https://api.tudominio.com/api/v1' > .env
npm install && npm run build
# Servir la carpeta dist/ con nginx, Apache o cualquier CDN
```

Nota: El proyecto no incluye Docker ni CI/CD. Se recomienda configurar ambos para despliegues reproducibles en producción.

13.4 Limitaciones actuales de despliegue

La siguiente tabla describe las limitaciones actuales de la configuración de despliegue y el impacto que cada una tiene al ejecutar el sistema en un entorno de producción.

Limitación	Impacto en producción
Sin Docker	Configurar Python, Node y dependencias manualmente en cada servidor
Sin CI/CD	No hay validación automática ni despliegue automatizado al hacer push
Sin Nginx configurado	Backend expuesto en el puerto 8000 sin SSL/TLS
Sin systemd/supervisor	El proceso de uvicorn muere si el servidor se reinicia
Pydantic v1 en EOL	Mayor consumo de memoria; migración a v2 pendiente antes de producción
cryptography sin tope	Riesgo de breaking changes en actualizaciones automáticas de pip

13.5 Variables que deben de cambiar entre desarrollo y producción

La siguiente tabla lista las variables de entorno que deben configurarse de manera diferente entre el entorno de desarrollo local y el de producción.

Variable	Desarrollo	Producción
SECRET_KEY	Cualquier valor (emite warning)	Aleatorio criptográfico de 64 hex chars
DATABASE_URL / SQLITE	sqlite:///app.db	mysql+pymysql://user:pass@host/db
AUTO_CREATE_TABLES	true	false (usar alembic upgrade head)

SMTP_*	Vacío — modo demo	Credenciales SMTP reales
ACTIVATION_URL_BASE	http://localhost:5173/activate	URL pública del frontend
CORS_ORIGINS	http://localhost:5173	Dominio de producción HTTPS
FIELD_ENCRYPTION_KEY	Opcional (datos en claro)	OBLIGATORIO — Fernet.generate_key()
VITE_API_URL	http://localhost:8000/api/v1	https://api.tudominio.com/api/v1

14. Seguridad

14.1 Mecanismos implementados

La siguiente tabla resume los mecanismos de seguridad implementados en el sistema, el área que protegen y su estado actual de implementación.

Área	Mecanismo	Estado
Autenticación	JWT de dos pasos con TOTP RFC 6238 obligatorio	Productivo
Contraseñas	Hash bcrypt + política de contraseñas (lista de comunes prohibidas)	Productivo
Integridad de datos	Hash SHA-256 en cada registro de migrante	Productivo
Cifrado de campos	Fernet (AES-128-CBC + HMAC-SHA256) sobre name_or_alias y contact_info	Opcional (env var)
Certificados de identidad	RSA-2048, X.509 por usuario	Productivo
Firma documental	ECDSA P-256 en el navegador (Web Crypto API); clave privada nunca en servidor	Productivo
TOTP secreto	Almacenado cifrado con Fernet en credentials.totp_secret_enc	Productivo
CORS	Restringido a orígenes configurados (CORS_ORIGINS)	Productivo
Tokens de activación	SHA-256 almacenado, un solo uso, expiran en 48h	Productivo

Auditoría	Log inmutable de todas las acciones críticas con IP, actor y hash	Productivo
-----------	---	------------

14.2 Riesgos conocidos y recomendaciones

La siguiente tabla detalla los riesgos de seguridad identificados en el sistema, su nivel de severidad y las recomendaciones para mitigarlos antes de ir a producción.

#	Riesgo	Severidad	Recomendación
1	JWT almacenado en localStorage (vulnerable a XSS)	Alta	Migrar a httpOnly cookie en producción
2	Migraciones 0002/0003 usan PRAGMA SQLite, no compatibles con MySQL	Alta	Reescribir migraciones para MySQL en producción
3	Hard delete de registros de migrantes sin soft delete	Media	Implementar campo deleted_at para borrado lógico
4	Pydantic v1 (deuda técnica)	Media	Migrar a Pydantic v2 antes de siguiente sprint
5	Doble almacenamiento de campos sensibles (claro + cifrado)	Media	Limpiar campos en claro una vez validado el cifrado
6	Links de activación en memoria (se pierden al reiniciar)	Baja	Persistir en BD o usar SMTP en producción
7	Sin rate limiting en /auth/login	Media	Agregar slowapi o middleware de throttling para prevenir fuerza bruta
8	JWT sin blacklist (no hay revocación)	Media	Implementar blacklist en Redis o reducir ACCESS_TOKEN_EXPIRE_MINUTES
9	Folio de bitácora no atómico (SELECT MAX + INSERT)	Media	Usar AUTOINCREMENT/SEQUENCE en BD; aplicar SELECT FOR UPDATE en MySQL
10	Generación de folios de registro no transaccional	Baja	Contador atómico en BD para CM-YYYY-NNNN, ARCO-YYYY-NNNN y CM-USR-NNNN
11	Demo links en memoria: se pierden al reiniciar	Media	No usar modo demo en producción; configurar SMTP real

14.3 Generación de claves para producción

```
# Generar SECRET_KEY
python -c "import secrets; print(secrets.token_hex(32))"

# Generar FIELD_ENCRYPTION_KEY
python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
```

14.4 Buenas prácticas ya aplicadas en el código

- CORS configurado explícitamente — solo orígenes en CORS_ORIGINS son aceptados
- Verificación bidireccional de firma: soporta DER e IEEE P1363 (64 bytes raw de WebCrypto)
- ARCO con anonimización controlada — no borrado inmediato, proceso con doble aprobación
- Bitácora de auditoría de todas las operaciones críticas con IP, actor y hash
- Validación de vigencia expires_at en cada request, no solo en login
- Claim stage en JWT — impide uso del token pre-2FA para recursos protegidos
- Hash SHA-256 de tokens de activación — no el token en claro en la BD
- La clave privada ECDSA no se almacena en el backend (signing_service.py:148-149)
- bcrypt para contraseñas (no MD5/SHA1 ni SHA-256 directo)

15. Guía de Contribución

15.1 Convenciones de código

- Backend: seguir PEP 8, nombres en snake_case, docstrings en funciones públicas.
- Frontend: componentes React en PascalCase, hooks en camelCase con prefijo use.
- Los nuevos módulos de API deben seguir la estructura: router → service → repository.
- Toda acción crítica (CRUD en registros, cambios de rol, revocación) debe registrarse en audit_logs.

15.2 Convenciones de migraciones Alembic

- Nombrar las migraciones con formato: YYYYMMDD_NNNN_descripcion_corta.
- Verificar compatibilidad MySQL/SQLite antes de usar PRAGMA o sintaxis específica.
- Las migraciones deben ser idempotentes en operaciones de seed.

15.3 Flujo de trabajo recomendado

- Crear una rama por funcionalidad: feature/nombre-funcionalidad.
- Ejecutar pytest antes de cualquier merge.
- Actualizar el .env.example si se agregan nuevas variables de entorno.
- Documentar los nuevos endpoints en el README y en este SDK.

15.3b Cómo agregar un nuevo endpoint (paso a paso)

Regla: toda acción crítica (CRUD en registros, cambios de estado, revocaciones) debe registrarse en `audit_logs` con `actor`, `resource`, `result` y `detail`.

Paso 1: Crear schema en `schemas/` si es necesario (clase Pydantic con campos `request/response`).

Paso 2: Agregar lógica en el servicio correspondiente (`services/`). Usar métodos `@staticmethod` que reciben `db: Session`.

Paso 3: Agregar endpoint en el router (`api/routes/`). Incluir Depends de seguridad y llamada a `AuditService.log()`.

Paso 4: Los routers ya están incluidos en `api/router.py` — no requieren re-registro.

Paso 5: Agregar wrapper en el frontend en `api/dominioApi.js`: `export const accion = (params) => client.método(url, body)`.

15.4 Tareas pendientes (siguiente sprint)

La siguiente tabla lista las tareas de desarrollo pendientes para el siguiente sprint, agrupadas por módulo e indicando la prioridad de cada una.

Módulo	Tarea	Prioridad
documents	Implementar CRUD de documentos (actualmente placeholder)	Alta
verification	Implementar verificación pública de folio/hash	Alta
Seguridad	Migrar JWT a <code>httpOnly</code> cookie	Alta
BD	Reescribir migraciones 0002/0003 para compatibilidad MySQL	Alta
Registros	Implementar soft delete (campo <code>deleted_at</code>)	Media
Backend	Migrar Pydantic v1 → v2	Media
Tests	Ampliar cobertura de pruebas unitarias en <code>services</code> y <code>repositories</code>	Media
Infra	Configurar Docker y CI/CD	Media
Datos	Limpiar campos en claro una vez validado el cifrado Fernet	Media
Seguridad	Hacer <code>SECRET_KEY</code> obligatorio (lanzar error si tiene valor default)	Alta
Backend	Agregar rate limiting en <code>/auth/login</code> con <code>slowapi</code> o similar	Alta
BD	Hacer generación de folios atómica (<code>SELECT FOR UPDATE</code> o <code>SEQUENCE</code>)	Media
Frontend	Corregir typo: <code>audutApi.js</code> → <code>auditApi.js</code>	Baja

Pruebas	Ampliar tests a AuthService, RecordService, ArcoService, SigningService	Alta
---------	---	------

16. Problemas Conocidos y Limitaciones

Esta sección documenta las limitaciones técnicas identificadas en el código fuente del proyecto, junto con su impacto y posibles alternativas.

La siguiente tabla documenta las limitaciones técnicas identificadas en el código fuente, el impacto que generan en la operación y si existe algún workaround disponible.

Área	Limitación	Impacto	¿Tiene workaround?
Seguridad / JWT	SECRET_KEY viene con valor por defecto "change-this-secret" y solo emite warnings.warn, sin detener el arranque. (backend/app/core/config.py:43-50)	En producción, cualquier servicio podría forjar tokens JWT válidos.	Sí: configurar SECRET_KEY en .env antes del despliegue con: python -c "import secrets; print(secrets.token_hex(32))"
Cifrado de campos	FIELD_ENCRYPTION_KEY es completamente opcional; si no se configura, name_or_alias_enc y contact_info_enc se almacenan sin cifrar. (backend/.env.example:75-77)	Datos personales de migrantes quedan en texto plano en la BD.	Sí: generar clave Fernet e incluirla en .env.
Base de datos — SQLite en producción	El default es SQLite (sqlite:///app.db). SQLite no soporta escrituras concurrentes verdaderas ni múltiples workers. (backend/app/core/config.py:99-109)	Con más de un proceso uvicorn/gunicorn aparecen OperationalError: database is locked.	Sí: migrar a MySQL con las variables MYSQL_* o DATABASE_URL.
SMTP / correo	Sin SMTP_HOST/USER/PASSWORD, el sistema arranca en "modo demo": los enlaces de activación NO se envían, quedan en memoria RAM. (backend/app/main.py:21-32)	En producción, ningún colaborador recibirá su enlace de activación.	Sí: modo demo expone GET /api/v1/auth/demo/activation-links para recuperar enlaces manualmente.
Módulo verification	El router verification.py devuelve únicamente un placeholder. La verificación pública de folios/hashes no está	No es posible verificar externamente la autenticidad de un registro sin acceso admin.	No: pendiente para sprint 2.

Área	Limitación	Impacto	¿Tiene workaround?
	implementada. (backend/app/api/routes/verification.py:6-12)		
Módulo documents	El router documents.py es un placeholder explícito. (backend/app/api/routes/documents.py:8-14)	La gestión documental completa (adjuntar archivos a registros) no existe.	No: pendiente sprint 2.
Configuración de despliegue	No existen archivos Dockerfile, docker-compose.yml, ni configuración de Nginx/Gunicorn/HostGator en el repositorio.	Un desarrollador nuevo no puede desplegar a producción sin construir la infraestructura desde cero.	Parcial: el README describe pasos manuales con uvicorn; se necesita añadir archivos de infraestructura.
Escalabilidad de la bitácora	La consulta de auditoría usa un tope duro de 500 registros, sin paginación por cursor. (backend/app/api/routes/audit.py:34)	Con volumen alto de eventos, el historial temprano queda inaccesible desde la API.	No: no hay paginación offset ni cursor implementada.
Folio de bitácora no atómico	_generate_audit_folio() busca el último BIT-NNNNN y suma 1 con Python, sin bloqueo a nivel BD. (backend/app/services/audit_service.py:10-25)	Con concurrencia alta, dos hilos pueden generar el mismo folio simultáneamente.	Parcial: SQLite serializa escrituras; en MySQL se necesita una secuencia o SELECT FOR UPDATE.
Token de pre-2FA en memoria	Los tokens pre_2fa y los códigos OTP en modo demo se almacenan en listas Python en memoria. (backend/app/services/notification_service.py:11-12)	Al reiniciar el servidor se pierden todos los tokens/códigos pendientes; incompatible con múltiples workers.	No: no hay Redis ni caché distribuida configurada.
Compatibilidad ECDSA frontend/backend	Los parámetros PBKDF2 (PBKDF2_ITERATIONS=600_000, SALT_LENGTH=16, IV_LENGTH=12) deben coincidir exactamente entre signing_service.py y webCrypto.js.	Un cambio en uno sin actualizar el otro rompe toda la verificación de firmas.	No hay test de integración automatizado para esta sincronización.
Contraseña admin en .env.example	El ejemplo usa ADMIN_PASSWORD=Admin123! como valor por defecto. (backend/.env.example:81)	Si se usa en producción sin cambiar, la cuenta admin queda con contraseña conocida públicamente.	Sí: cambiar en .env antes del primer alembic upgrade head.

Área	Limitación	Impacto	¿Tiene workaround?
Vigencia de certificados firmantes	CERT_YEARS_DEFAULT = 1: los certificados ECDSA expiran tras 1 año con revocación automática al generar uno nuevo. (backend/app/services/signing_service.py:49)	Si un usuario no renueva, pierde capacidad de firma sin aviso.	Parcial: hay endpoint de revocación explícito; no hay notificación proactiva de expiración.
Roles / permisos sin CRUD de UI	El módulo roles.py existe en el backend, pero no hay página de administración de roles en el frontend.	Los permisos granulares solo se pueden modificar directamente en BD.	No existe endpoint público para gestión de permisos desde la interfaz.
vite.config.js — host 0.0.0.0	El servidor de desarrollo de Vite escucha en todas las interfaces (host: "0.0.0.0"). (frontend/vite.config.js:7)	Cualquier dispositivo en la misma red local puede acceder al frontend de desarrollo.	Sí: solo usar en entornos controlados; en producción usar npm run build.
Concurrencia — folios duplicados	Los generadores de folios usan SELECT MAX + 1 sin bloqueo. Con múltiples workers en MySQL pueden duplicarse.	Con carga alta pueden aparecer folios duplicados. En SQLite no ocurre.	Parcial: usar un solo worker uvicorn, o migrar a SEQUENCE con prefijo en aplicación.
DateTime SQLite vs MySQL	SQLite almacena DATETIME sin timezone. El código normaliza con replace(tzinfo=utc) en varios servicios.	Código defensivo que puede enmascarar errores en la migración a MySQL.	Sí: en MySQL el driver PyMySQL retorna objetos aware y el workaround se vuelve innecesario.
Contraseña admin hardcodeda en .env.example	ADMIN_PASSWORD=Admin123! como default. Si el seed corre sin cambiar .env, el admin queda vulnerable.	Cuenta admin con contraseña pública conocida en cualquier entorno que no actualizó el .env.	Sí: cambiar ADMIN_PASSWORD en .env antes del primer reset_and_seed.py.
cryptography sin versión máxima fijada	requirements.txt especifica cryptography>=42.0.0 sin límite superior.	Una actualización de pip puede traer breaking changes y romper signing_service.py.	Sí: fijar como cryptography>=42.0.0,<44.0.0 y probar al actualizar.
Roles y permisos sin UI de administración	El backend expone /roles pero no hay página frontend para gestionar permisos granulares.	Los 11 permisos solo se pueden modificar directamente en la base de datos.	No: pendiente implementar página de administración de roles en el frontend.

17. Casos de Uso Principales

CU-01: Autenticación con doble factor (2FA / TOTP)

Actor: Cualquier colaborador registrado (niveles 1–4).

Precondición: El colaborador tiene una cuenta ACTIVE y ya completó el enrolamiento TOTP.

Flujo principal:

- El usuario envía POST /api/v1/auth/login con email y password vía application/x-www-form-urlencoded.
- auth_service.py verifica contraseña con bcrypt, comprueba status == ACTIVE y expires_at.
- El backend devuelve un pre-2FA token (stage: "pre_2fa") con requires_totp_enrollment: false.
- El frontend redirige a la pantalla de ingreso de código TOTP.
- El usuario abre su app de autenticación y obtiene el código de 6 dígitos.
- El usuario envía POST /api/v1/auth/verify-2fa con el token pre-2FA y el código TOTP.
- totp_service.py descifra el secreto TOTP con Fernet y verifica el código con tolerancia de ± 30 s.
- Si el código es válido, el backend emite el JWT final (stage: "authenticated", exp. 60 min por defecto).
- El frontend guarda el JWT en localStorage y AuthContext y redirige al Dashboard.
- Todas las rutas protegidas validan stage == "authenticated" vía require_active_user en deps.py.

Flujo alternativo / error:

- Contraseña incorrecta: HTTP 401; evento auth.login con result=FAILURE registrado en bitácora.
- Cuenta inactiva o expirada: HTTP 403 — "La cuenta ha expirado. Contacte al administrador."
- Código TOTP inválido: HTTP 401; evento auth.2fa.verify con FAILURE registrado en bitácora.
- Token pre-2FA usado en endpoint protegido: HTTP 403 — "Token de pre-autenticación. Complete el segundo factor."

Postcondición: El JWT válido permite al usuario acceder a los recursos permitidos por su nivel de acceso durante 60 minutos (configurable con ACCESS_TOKEN_EXPIRE_MINUTES).

CU-02: Enrolamiento inicial de TOTP (primer inicio de sesión)

Actor: Colaborador recién activado (cualquier nivel), sin TOTP configurado.

Precondición: El colaborador activó su cuenta vía enlace de activación y tiene totp_enrolled = false.

Flujo principal:

- El usuario completa el login de credenciales (CU-01, pasos 1–3); el backend devuelve requires_totp_enrollment: true.
- El frontend redirige a /security/totp (TOTPSetupPage).

- Con el token pre_2fa, el usuario llama POST /api/v1/auth/totp/enroll/begin.
- totp_service.py genera un secreto base32 aleatorio y construye un QR PNG en base64.
- El frontend muestra el QR y el secreto en texto para ingreso manual.
- El usuario escanea el QR con su app de autenticación.
- El usuario ingresa el código de 6 dígitos y envía POST /api/v1/auth/totp/enroll/confirm.
- El backend verifica el código; si es válido, cifra el secreto con Fernet y lo persiste; marca totp_enrolled = true.
- El backend emite el JWT final (stage: "authenticated") — el usuario queda autenticado completamente.

Flujo alternativo / error:

- Código de confirmación incorrecto: HTTP 400 o 401; el secreto NO se persiste; el usuario puede reintentar.
- Token pre-2FA expirado (10 min por defecto): HTTP 401; el usuario debe volver a hacer login completo.
- Enrolamiento ya completado: el endpoint rechaza con error si totp_enrolled = true ya existe.

Postcondición: El usuario tiene 2FA habilitado de forma permanente; en todos los logins futuros deberá ingresar su código TOTP.

CU-03: Alta de colaborador

Actor: Administrador (nivel 1).

Precondición: Sesión autenticada con JWT stage: "authenticated" y access_level.code == 1.

Flujo principal:

- El administrador navega a /admin/users (UsersAdminPage).
- Completa el formulario: nombre, email, área, nivel de acceso (1–4), rol y fechas de vigencia.
- Envía POST /api/v1/identity/collaborators.
- IdentityService.create_collaborator() verifica que el email no exista (HTTP 409 si existe).
- El servicio genera una matrícula única con formato CM-USR-{seq:04d}.
- Crea el usuario con status = PENDING_ACTIVATION y genera token de activación (hash SHA-256 en ActivationToken).
- CryptoService emite un certificado RSA-2048 de identidad (X.509) persistido en Certificate con cert_type = "IDENTITY".
- NotificationService envía el enlace de activación por SMTP; si falla, lo almacena en modo demo.
- El evento se registra en bitácora: identity.create_collaborator con result=SUCCESS.

Flujo alternativo / error:

- Email duplicado: HTTP 409 — "El correo ya existe".
- Área inexistente: HTTP 404 — "Área no encontrada".
- SMTP no configurado: el enlace no se envía por correo; el admin puede recuperarlo vía GET /api/v1/auth/demo/activation-links.

Postcondición: El usuario existe en BD con status = PENDING_ACTIVATION; recibirá un enlace válido por ACTIVATION_TOKEN_EXPIRE_HOURS horas (default 48 h) para establecer su contraseña y activar la cuenta.

CU-04: Registro de migrante

Actor: Operativo (nivel 3), Coordinador (nivel 2), Administrador (nivel 1) o Voluntario externo (nivel 4).

Precondición: Sesión autenticada; el actor tiene nivel ≤ 4 .

Flujo principal:

- El usuario navega a /records/new (IntakeFormPage).
- Completa el formulario: nombre/alias, fecha de atención, país de origen, necesidades, observaciones, área, entre otros.
- Envía POST /api/v1/records.
- RecordService genera un folio único CM-{año}-{seq:04d}.
- Si FIELD_ENCRYPTION_KEY está configurada, name_or_alias y contact_info se cifran con Fernet y se almacenan en campos _enc.
- Se calcula el SHA-256 del diccionario de campos y se persiste en sha256_hash.
- El registro se crea con workflow_status = "pendiente" y assigned_operator_id = actor.id.
- El evento records.create se registra en bitácora con el hash.

Flujo alternativo / error:

- Campos requeridos faltantes: HTTP 422 (Pydantic ValidationError) con detalles por campo.
- Token expirado durante captura: HTTP 401; el formulario no se envía y los datos no guardados se pierden.

Postcondición: El registro existe en BD con folio único, integridad SHA-256 calculada y workflow_status = "pendiente"; visible en el worklist del coordinador en el dashboard.

CU-05: Flujo operativo de canalización de registro

Actor: Operativo (nivel 3) y Coordinador (nivel 2).

Precondición: Existe un registro con workflow_status = "pendiente" creado por el operativo.

Flujo principal:

- El operativo accede al dashboard y abre el detalle del registro en /records/:id.

- Agrega notas de canalización y ejecuta PUT /api/v1/records/{id}/channel.
- El backend verifica que el actor sea nivel 3 y el registro tenga workflow_status = "pendiente".
- El sistema cambia workflow_status = "canalizado", registra channeled_at = now() y asigna al coordinador del área.
- El evento se registra en bitácora: records.channel.
- El coordinador ejecuta PUT /api/v1/records/{id}/review — cambia workflow_status = "revisado".
- El coordinador puede cerrar con PUT /api/v1/records/{id}/close — cambia workflow_status = "cerrado".

Flujo alternativo / error:

- El registro ya fue canalizado: HTTP 400 — el servicio valida transiciones de estado.
- Operativo intenta cerrar o editar: HTTP 403 — operaciones de cierre/edición restringidas a niveles ≤ 2 .

Postcondición: El registro avanza en el ciclo pendiente → canalizado → revisado → cerrado; cada transición queda en bitácora con timestamp y actor.

CU-06: Firma digital de documentos (modelo SAT)

Actor: Coordinador (nivel 2) o Administrador (nivel 1).

Precondición: El usuario tiene un certificado ECDSA P-256 vigente (status = "VALID", cert_type = "SIGNING") y el archivo .key descargado al momento de la emisión.

Flujo principal:

- El usuario navega a la acción que requiere firma (ej. atender solicitud ARCO, editar registro).
- El frontend muestra SignaturePanel; el usuario carga su archivo .key y digita la contraseña del certificado.
- webCrypto.js usa WebCrypto SubtleCrypto para descifrar la clave privada en memoria del navegador (PBKDF2 + AES-256-GCM con 600,000 iteraciones).
- El frontend construye un objeto JSON canónico con los campos del recurso a firmar.
- SubtleCrypto.sign("ECDSA", privateKey, data) produce la firma ECDSA P-256 en formato P1363 (64 bytes raw).
- El frontend codifica la firma en Base64 y envía POST /api/v1/auth/signing/verify con { resource_type, resource_id, content_hash, signature_b64 }.
- El backend carga la clave pública del certificado registrado del usuario y verifica la firma con la librería cryptography de Python.
- Si es válida, persiste la evidencia en DocumentSignature y retorna { is_valid: true }.

Flujo alternativo / error:

- Contraseña del .key incorrecta: el descifrado en el navegador falla; la clave privada nunca llega al backend.

- Firma inválida: el backend retorna { is_valid: false }; la acción principal se bloquea.
- Certificado revocado o expirado: HTTP 403 al verificar cert.status != "VALID".
- Usuario sin certificado: GET /api/v1/auth/signing/status retorna { enrolled: false }; debe ir a /security/signing para generarlo.

Postcondición: La evidencia criptográfica queda en DocumentSignature vinculada al recurso y al certificado; cualquier alteración posterior del contenido se detectaría al reverificar.

CU-07: Solicitud y resolución ARCO de Cancelación

Actor: Operativo (nivel 3) → Coordinador (nivel 2) → Administrador (nivel 1).

Precondición: Existe un registro de migrante; no hay otra solicitud ARCO activa para ese registro.

Flujo principal:

- El operativo crea la solicitud: POST /api/v1/arco-requests con request_type = "CANCELLATION", record_id y reason.
- ArcoService genera folio ARCO-{año}-{seq:04d}, establece status = "PENDING" y auto-asigna coordinador del área.
- El coordinador intenta atender mediante PUT /api/v1/arco-requests/{id}/attend.
- El backend valida que las solicitudes de tipo CANCELLATION no pueden ser completadas por el coordinador y devuelve instrucción de escalar.
- El coordinador escala con PUT /api/v1/arco-requests/{id}/escalate; el estado pasa a "ESCALATED".
- El administrador revisa la solicitud con PUT /api/v1/arco-requests/{id}/review.
- El admin proporciona firma digital (ver CU-06) + resolution ("APPROVED" o "REJECTED") + resolution_notes.
- Si APPROVED: ArcoService anonimiza controladamente — se nulifican los campos identificables, se recalcula sha256_hash y se marcan is_anonymized = true.
- El evento se registra en bitácora: arco.review con resultado y detalles.

Flujo alternativo / error:

- Ya existe solicitud activa para el registro: HTTP 409.
- Coordinador intenta aprobar directamente: HTTP 403 — regla de negocio implementada en el servicio.
- Firma inválida del administrador: la resolución no se procesa; la solicitud permanece en ESCALATED.
- Rechazo (REJECTED): el registro no se modifica; la solicitud cierra con status = "REJECTED".

Postcondición: Los datos personales identificables del registro quedan en NULL (si fue aprobado); is_anonymized = true. El registro sigue existiendo en BD para efectos estadísticos pero no puede re-identificarse.

CU-08: Consulta de bitácora de auditoría

Actor: Administrador (nivel 1).

Precondición: Sesión autenticada con nivel 1.

Flujo principal:

- El administrador navega a /audit (AuditPage).
- El frontend llama GET /api/v1/audit/ con JWT.
- El backend verifica `access_level.code == 1`; si es `nivel > 1` retorna HTTP 403.
- Se retornan hasta 500 entradas (configurable con `?limit=N`) de la tabla `audit_log`, ordenadas por id DESC.
- Opcionalmente se puede filtrar por matrícula del actor: GET /api/v1/audit/?matricula=CM-USR-0001.
- Cada entrada incluye: folio (BIT-NNNNN), `actor_user_id`, `action`, `resource`, `result` (SUCCESS/FAILURE), `detail`, `hash_related`, `ip_address`, `created_at`.
- El propio acceso a la bitácora genera un evento `audit.list` en la bitácora.

Flujo alternativo / error:

- Usuario `nivel > 1` intenta acceder: HTTP 403 — "Acceso a bitácora restringido al administrador del sistema".
- Sin registros: retorna lista vacía [].
- Matrícula sin eventos: retorna lista vacía para ese filtro.

Postcondición: El administrador visualiza el historial de eventos; la consulta misma queda registrada en la bitácora (inmutabilidad por diseño — no hay endpoint de eliminación de logs).

18. FAQ Técnica y Registro de Errores (LOGS)

18.1 FAQ Técnica

P1: ¿Es crítico el warning "SECRET_KEY no ha sido configurado con un valor seguro" al arrancar el backend?

Causa probable: La variable `SECRET_KEY` tiene el valor por defecto "change-this-secret". En desarrollo es inocuo; en producción permite forjar JWTs válidos. (backend/app/core/config.py:43-50)

Solución:

```
# 1. Generar clave segura
python -c "import secrets; print(secrets.token_hex(32))"
# 2. Copiar el resultado al .env del backend
SECRET_KEY=<resultado del comando anterior>
# 3. Reiniciar el servidor
uvicorn app.main:app --reload
```

P2: El sistema arranca pero ningún colaborador recibe el email de activación.

Causa probable: SMTP no configurado: el sistema opera en "modo demo" y guarda los enlaces en memoria RAM. (backend/app/main.py:21-32)

Solución:

```
# Opción A – Recuperar enlace en modo demo (solo pruebas)
GET /api/v1/auth/demo/activation-links    (requiere autenticación de admin)
# Opción B – Configurar SMTP real (ejemplo Gmail)
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=tu-correo@gmail.com
SMTP_PASSWORD=tu-app-password-16-chars    (App Password, NO la contraseña de
cuenta)
SMTP_FROM_EMAIL=tu-correo@gmail.com
```

P3: alembic upgrade head falla con sqlite3.OperationalError: no such table.

Causa probable: La BD SQLite no existe todavía, o SQLITE_PATH apunta a un directorio incorrecto, o se ejecutó desde el directorio equivocado.

Solución:

```
# 1. Situar en la carpeta del backend
cd backend
# 2. Verificar SQLITE_PATH en .env
# Debe mostrar algo como ./app.db
# 3. Ejecutar la migración desde el directorio correcto
alembic upgrade head
```

P4: Al migrar de SQLite a MySQL en producción, algunas migraciones fallan.

Causa probable: Las migraciones usan guardas `_is_sqlite()` porque ciertas instrucciones DDL (ej. PRAGMA, AUTOINCREMENT) son exclusivas de SQLite.

Solución:

```
# 1. Configurar MySQL antes de la primera migración
DATABASE_URL=mysql+pymysql://usuario:contraseña@host:3306/casa_monarca
# 2. Crear la base de datos en MySQL
mysql -u root -p -e "CREATE DATABASE casa_monarca CHARACTER SET utf8mb4;"
# 3. Correr migraciones
cd backend && alembic upgrade head
# 4. Verificar que pymysql esté instalado
pip install pymysql
```

P5: El frontend muestra "Network Error" o "CORS policy" al llamar al backend.

Causa probable: CORS_ORIGINS en el backend no incluye el origen del frontend, o VITE_API_URL apunta a la URL incorrecta.

Solución:

```
# backend/.env – agregar el origen exacto (con protocolo y puerto)
CORS_ORIGINS=http://localhost:5173,http://127.0.0.1:5173,https://tu-dominio.
com
# frontend/.env
VITE_API_URL=http://localhost:8000/api/v1
```

- Reiniciar ambos servidores después de cualquier cambio en .env.

P6: La firma ECDSA falla con "Firma inválida" aunque se usa el archivo .key correcto.

Causa probable: Causa A: El número de iteraciones PBKDF2 en webCrypto.js no coincide con PBKDF2_ITERATIONS = 600,000 del backend. Causa B: El certificado activo fue revocado al generar uno nuevo. (backend/app/services/signing_service.py:43)

Solución:

```
# Verificar estado del certificado
GET /api/v1/auth/signing/status
```

- Si enrolled=false o status="REVOKED", re-emitir con:
POST /api/v1/auth/signing/enroll (body: { key_password: "tu-contraseña" })
- Descargar el nuevo .key y .cer.
- Si es problema de parámetros: verificar que frontend/src/Utils/webCrypto.js
- y backend/app/services/signing_service.py tengan las mismas constantes (PBKDF2_ITERATIONS, SALT_LENGTH, IV_LENGTH).

P7: GET /api/v1/auth/verify-2fa devuelve 403 con "Token de pre-autenticación. Complete el segundo factor."

Causa probable: El cliente está usando el token pre-2FA (emitido por /login) para acceder a un endpoint protegido en lugar del JWT final (emitido por /verify-2fa).

Solución:

- El flujo correcto es siempre: POST /login → guardar token pre-2FA temporalmente
- → POST /verify-2fa → reemplazar token en localStorage con el JWT final.
- Verificar en authApi.js que al hacer verify-2fa se reemplaza el token:
- `const { access_token } = await verifyTwoFactor(pre2fa_token, code);`
- `localStorage.setItem("token", access_token);` // sobrescribir el pre-2fa

P8: FIELD_ENCRYPTION_KEY estaba vacío en producción y ahora tiene valor. Los registros anteriores no se pueden descifrar.

Causa probable: Los campos cifrados fueron almacenados en texto plano porque FIELD_ENCRYPTION_KEY estaba vacío; ahora el servicio intenta descifrarlos con Fernet y falla.

Solución:

- La clave Fernet es simétrica: debe usarse la misma con la que se cifraron los datos.
- Si los datos están en texto plano, FieldEncryptionService.decrypt_or_none() tiene lógica de fallback.
- Para migrar datos existentes a cifrado, se requiere un script de migración (no incluido):
Backup primero
- `cp app.db app.db.bak`
- Luego correr el script de re-encryptación (debe crearse)
- Generar la clave correctamente:
from cryptography.fernet import Fernet
print(Fernet.generate_key().decode()) # guardar en FIELD_ENCRYPTION_KEY

P9: Al activar la cuenta, el enlace ya expiró.

Causa probable: ACTIVATION_TOKEN_EXPIRE_HOURS (default 48 h) expiró antes de que el colaborador activara su cuenta. (backend/app/core/config.py:71)

Solución:

- El administrador debe revocar el token expirado y crear de nuevo al colaborador para generar un nuevo enlace.
- Para aumentar el tiempo de validez del enlace:
`ACTIVATION_TOKEN_EXPIRE_HOURS=168 # 7 días`

18.2 Sistema de Logs y Registro de Errores

El sistema cuenta con dos capas de logging independientes:

Capa 1 — Python logging estándar (consola/stderr)

Todos los módulos que generan eventos externos usan el patrón estándar: `logger = logging.getLogger(__name__)`. Los módulos con logging activo son:

- backend/app/main.py:12 — advertencia de SMTP al arrancar.
- backend/app/services/notification_service.py:9 — errores SMTP en detalle (auth, connect, send, OS).
- backend/app/services/notification_service.py:62 — modo demo activo.

Errores SMTP capturados explícitamente: `SMTPAuthenticationError`, `SMTPConnectError`, `SMTPException` y `OSError` (red) — todos registrados con `logger.error()`. (notification_service.py:45-59)

Capa 2 — Bitácora de auditoría en base de datos (AuditService)

`AuditService.log()` persiste eventos en la tabla `audit_log`. (backend/app/services/audit_service.py:28-56)

La siguiente tabla describe las acciones registradas por AuditService en la bitácora del sistema, los posibles resultados de cada acción y el momento en que se generan.

Acción (action)	Result posible	Cuándo
auth.login	SUCCESS / FAILURE	Cada intento de login
auth.2fa.verify	SUCCESS / FAILURE	Verificación TOTP
auth.totp.enroll	SUCCESS	Enrolamiento TOTP completado
identity.create_collaborator	SUCCESS	Alta de usuario

Acción (action)	Result posible	Cuándo
identity.change_level	SUCCESS	Cambio de nivel de acceso
identity.toggle_status	SUCCESS	Activar/desactivar usuario
records.create	SUCCESS	Crear registro de migrante
records.update	SUCCESS	Editar registro (con firma)
records.channel	SUCCESS	Canalización operativo→coordinador
arco.create	SUCCESS	Crear solicitud ARCO
arco.escalate	SUCCESS	Escalar CANCELLATION a admin
arco.review	SUCCESS / FAILURE	Admin resuelve CANCELLATION
signing.verify	SUCCESS / FAILURE	Verificación de firma ECDSA
signing.enroll	SUCCESS	Emisión de certificado de firma
signing.revoke	SUCCESS	Revocación de certificado
audit.list	SUCCESS	Consulta de la propia bitácora

Niveles de log utilizados

La siguiente tabla describe los niveles de log utilizados en el proyecto y los eventos registrados en cada uno.

Nivel	Uso en el proyecto
logger.info(...)	Confirmación de envío exitoso de correo SMTP. (notification_service.py:42, 105)
logger.warning(...)	SMTP no configurado al arrancar (main.py:28-32); modo demo activo (notification_service.py:62-68).
logger.error(...)	Todos los errores SMTP capturados con excepción específica. (notification_service.py:45-59)

El nivel de log de uvicorn por defecto es INFO (configurable con --log-level).

Almacenamiento de logs

La siguiente tabla describe los diferentes tipos de logs del sistema, su destino de almacenamiento y su nivel de persistencia.

Tipo	Destino	Persistencia
Logs Python (logging)	stderr del proceso uvicorn — consola del terminal	Volátil (no hay rotación de archivos configurada)
Logs de uvicorn HTTP	stdout — cada request: método, URL, status, tiempo	Volátil
Eventos de auditoría (AuditService)	Tabla audit_log en la base de datos (SQLite o MySQL)	Persistente, inmutable (no hay DELETE en la API)

Nota: No existe configuración de logging a archivo (FileHandler, RotatingFileHandler) ni integración con sistemas externos (Sentry, Datadog, CloudWatch) en el código actual.

Consulta de la bitácora vía API

Endpoint exclusivo para administradores (nivel 1). HTTP 403 para cualquier otro nivel. (backend/app/api/routes/audit.py:26-32)

```
GET /api/v1/audit/                # todos los eventos recientes
(máximo 500)
GET /api/v1/audit/?matricula=CM-USR-0001  # filtrar por matrícula
GET /api/v1/audit/?limit=50           # limitar resultados
```

Recomendaciones para logging en producción

- Configurar logging a archivo con rotación usando RotatingFileHandler (maxBytes=10_000_000, backupCount=5) en backend/app/main.py.
- Configurar uvicorn con --access-log y --log-config logging_config.json para registrar cada request HTTP a archivo.
- Migrar a MySQL en producción para que la bitácora de auditoría sea duradera, sobreviva reinicios y soporte backups estándar (mysqldump).
- No exponer el endpoint de demo (/auth/demo/activation-links) en producción — configurar SMTP real. (notification_service.py:136-141)
- Añadir alertas sobre intentos fallidos de 2FA: la bitácora registra result=FAILURE con action=auth.2fa.verify; se puede implementar un cron que consulte audit_log y alerte si supera un umbral.
- Añadir índice compuesto en (actor_user_id, created_at) y paginar con offset para auditoría en BD con alto volumen de eventos. ([audit.py:34](#))

Falta Figuras de Apoyo

Descripción de tablas